Agentic AI Robotic Puzzle-Solver

An End-to-End Perception-Reasoning-Action Framework

Venkata Naga Sai Bhavan Koppolu

Arizona State University

ABSTRACT

This report documents the design, implementation, and evaluation of a three-stage Agentic AI Robotic Puzzle-Solver built on a Dobot Magician Lite robotic arm. The system progressively advances through three autonomous tasks, Tic-Tac-Toe (Minimax AI), 4×4 Maze Solving (Depth-First Search), and natural-language-driven Pick-and-Place (Google Gemini 2.5 Flash), each sharing a unified Perception-Planning-Action (PPA) pipeline and a two-stage camera-to-robot calibration architecture. Computer vision (OpenCV) provides real-time workspace perception; classical and large-language-model agents provide planning; and precision kinematic control executes the resulting motion.

Keywords: *Dobot Magician Lite, Robotic Arm, Computer Vision, Agentic AI, Minimax, Depth-First Search, Vision-Language-Action (VLA), Google Gemini, LLM Function Calling, OpenCV, Affine Transformation*

1. Introduction

Autonomous robotic systems that can perceive, reason, and act in unstructured environments represent a central challenge in modern robotics. This project addresses that challenge through three progressively complex tasks, each built on a shared Perception-Planning-Action (PPA) architecture deployed on a Dobot Magician Lite robotic arm with an end-effector-mounted camera.

The three tasks are unified by a common design philosophy: perception converts raw camera input into a structured digital representation of the world; planning (whether classical algorithm or large language model) reasons over that representation to produce an action sequence; and action executes the plan through calibrated kinematic control.

Task	Perception	Planning Agent	Action
Tic-Tac-Toe	Image Differencing	Minimax AI	Pen Drawing
Maze Solving	HSV Thresholding	Depth-First Search	Path Tracing
Pick-and-Place	HSV + JSON Scene	Gemini 2.5 Flash (LLM)	Suction Pick

2. Shared System Architecture

2.1 Hardware Setup

All three tasks run on identical hardware: a Dobot Magician Lite robotic arm connected via USB and controlled through the pydobot library in Python 3 on Windows 11. A USB camera mounted on the end-effector provides the visual input. For Tic-Tac-Toe a pen attachment handles drawing; for Pick-and-Place a suction cup handles object manipulation.

2.2 Two-Stage Calibration Pipeline

Precise spatial correspondence between camera image and physical workspace is the prerequisite for all three tasks. A two-stage process achieves this:

- **Stage 1 Perspective Correction:** A perspective transform matrix M is computed from a known planar target. Applied to every captured frame, it converts the camera's skewed view into a flat, top-down image of fixed resolution.
- **Stage 2 Pixel-to-Robot Mapping:** A homography (or affine) matrix H is computed by mapping known pixel coordinates to measured robot (x, y, z) coordinates, enabling any 2D pixel location to be converted directly to a 3D robot-space coordinate for execution.

Both matrices are persisted to disk as .npz files and reloaded at runtime. The calibration must be re-run if the robot base or workspace surface is moved.

2.3 Software Stack

The system is implemented entirely in Python 3 using: pydobot for robot control, opencv-python for all computer vision operations, numpy for matrix math, google-genai for LLM access (Task 3), and standard library modules for I/O and state management.

3. Task 1: Human vs. Robot Tic-Tac-Toe

3.1 Objective

Build a fully autonomous system in which the robot plays a complete game of Tic-Tac-Toe against a human opponent, handling every physical aspect: drawing the grid, detecting the human's move, choosing and drawing its own move, and writing the final result.

3.2 Perception: Image Differencing

Rather than classifying the board state directly, the system captures a reference image before each human turn and a comparison image after. Subtracting these with cv2.absdiff isolates exactly the new mark the human drew, yielding a single residual blob whose centroid is matched to the nearest of 9 pre-computed cell centres. This approach is robust to static lighting variation because it responds only to change, not absolute pixel intensity.

3.3 Planning: Minimax AI

The robot's move is selected by a Minimax algorithm that exhaustively evaluates every legal continuation and scores terminal states as win (+1), loss (−1), or draw (0). The algorithm is provably optimal: it will always block the human's winning move and take its own winning move when available, guaranteeing at least a draw from any position.

3.4 Action: Calibrated Drawing Library

All physical output uses two pre-calibrated Z-heights: DRAW_Z (pen on surface) and SAFE_Z (pen lifted). A drawing library implements grid drawing, X/O symbol rendering, winning line strike-through, and blocky stroke-based text for the final result.

3.5 Results

The system completed full end-to-end games reliably. The Minimax agent never lost. Image differencing detected all human moves correctly once the perception threshold was tuned for ambient lighting. The primary limitation is that calibration files must be regenerated if the robot or paper is repositioned.

4. Task 2: Autonomous 4×4 Maze Solver

4.1 Objective

Given a physical 4×4 maze placed on the workspace, the robot must scan the maze with its camera, digitise the wall structure, plan a solution path, and physically trace that path with its end-effector.

4.2 Perception: Grid Digitisation

The robot homes to a fixed PHOTO_HOME position to ensure a consistent overhead view. The perspective matrix M is applied to produce a 350×350 top-down image. The build_maze_grid function binarises the image with a fixed threshold to classify each of 16 cells as open or walled. Start and end markers are identified by HSV colour segmentation.

This fixed-threshold approach proved to be the system's primary vulnerability: minor changes in ambient lighting caused misclassified cells, producing a flawed digital map that the planner then correctly solved, yielding an incorrect physical path.

4.3 Planning: Depth-First Search

A standard iterative DFS explores the 4×4 cell graph from the start marker to the open edge (end). The resulting cell sequence is pruned to turning-point waypoints only, reducing unnecessary intermediate moves and producing a compact, executable path.

4.4 Action: Waypoint Tracing

Each waypoint pixel is passed through the homography matrix H to obtain a physical (x, y) coordinate. The robot lowers to MAZE_RUN_Z above the board surface and traces the path segment by segment. The action and calibration phases were fully successful, the robot moved smoothly and precisely to all planned coordinates.

4.5 Results and Lessons Learned

Planning (DFS) and Action (kinematic execution) performed reliably. The system's bottleneck was Perception: a static intensity threshold is not robust to lighting variation. The key takeaway is that a correct AI agent is useless if fed a corrupted world model. Future work would replace the fixed threshold with adaptive thresholding or edge-based wall detection.

5. Task 3: LLM-Driven Pick-and-Place (VLA Framework)

5.1 Objective

Enable the robot to execute arbitrary pick-and-place commands expressed in natural language (e.g., "place the blue block to the right of the red one") by integrating a Large Language Model as the reasoning agent in the PPA pipeline.

5.2 System Architecture: Agentic Loop

The core logic operates as a continuous Think-Act loop. The user submits a natural-language command; the Gemini 2.5 Flash model determines which Python tool functions to call and in what order; the tools execute perception or motion and return structured results to the model; the model reasons over the results and issues further tool calls or reports completion.

5.3 Perception: HSV Detection and Scene JSON

The capture_scene_with_detection tool captures a frame and runs detect_and_annotate_blocks, which applies HSV colour thresholding to identify blocks by colour (Blue, Red, Green, Yellow), computes each block's centroid, and serialises the result to capture_scene.json. The LLM reads this JSON to obtain the current spatial state of the workspace before planning any motion.

5.4 Reasoning: LLM Function Calling

Rather than hard-coding spatial logic, the Gemini model is provided a toolbox of Python functions and autonomously selects and sequences tool calls to fulfil the user's intent. Spatial prepositions such as "to the right of" and "beside" are parsed and resolved by the LLM into concrete placement_type arguments passed to the motion function. Prompt engineering of the system prompt proved as critical as the code itself.

5.5 Action: Affine Mapping and Suction Control

A 3×3 Affine Transform matrix maps 2D image pixel coordinates (u, v) to 3D robot coordinates (x, y). Z-height constants manage obstacle clearance and suction engagement depth. The motion sequence for each pick-and-place operation is: move to Safe Z → travel to source → descend → engage suction → ascend → travel to target → descend → release suction.

5.6 Error Handling

When the vision tool cannot detect a requested block, it returns a structured error string. The LLM correctly propagates this to the user rather than hallucinating a motion, a significant robustness property compared to hard-coded planners.

5.7 Results

The system successfully interpreted and executed commands involving six distinct spatial relations. Motion control was smooth and accurate across all tested configurations. Residual failure modes were limited to HSV detection errors under strong ambient light variation, the same root cause as in Task 2.

6. Cross-Task Analysis

6.1 Evolution of Perception

Across the three tasks, perception evolved from the most robust (Task 1: image differencing, immune to static scene variation) through the most fragile (Task 2: fixed intensity threshold) to a hybrid approach (Task 3: HSV colour segmentation with LLM-mediated error recovery). Perception quality is the dominant factor in end-to-end system reliability.

6.2 Evolution of Planning

Planning evolved from deterministic rule-based AI (Minimax, Task 1), through classical graph search (DFS, Task 2), to probabilistic semantic reasoning (Gemini LLM, Task 3). Each step increased generality at the cost of interpretability: Minimax is provably optimal; DFS is complete on finite graphs; the LLM generalises to arbitrary natural-language commands but introduces prompt-sensitivity and API latency.

6.3 Calibration Architecture

The two-stage calibration pipeline proved consistent and accurate across all three tasks. This shared foundation demonstrates that a single well-designed spatial calibration framework can support diverse task types, from drawing to path-tracing to manipulation, without task-specific modification.

7. Conclusion

This project demonstrates a complete Agentic AI robotic system built on a unified Perception-Planning-Action architecture. Across three tasks of increasing complexity-game playing, spatial navigation, and natural-language-driven manipulation, the same calibration framework and PPA design pattern were applied with consistent success. The dominant engineering lesson is the primacy of perception quality: even a provably optimal planner is only as effective as the world model it receives. The final VLA framework, where a large language model sequences tool calls to translate natural-language intent into physical robot motion, is the project's most significant contribution, showing how semantic reasoning can be composed with classical kinematic control to build flexible, instruction-following robotic systems.